

API Design in System Design Interview

 systemdesignschool.io/fundamentals/api-design-intro



API Design Intro

API design is the process of defining the interface between different software components. It involves specifying the endpoints, request/response formats, and the operations that can be performed on the API. It's a common question in system design interviews, especially for junior positions. Typically, you will be asked to design a RESTful API for a given scenario. GraphQL is another popular API design that is growing in popularity but most of the interview questions still focus on RESTful API design.

What is a REST API?

A REST API is a set of rules that developers follow when creating APIs. It relies on standard HTTP methods and uses resource-based URLs to perform operations. The core principles of RESTful APIs include:

- **Statelessness:** Each request from a client must contain all the information needed to process it, and the server doesn't store any session data.
- **Uniform Interface:** Resources are accessed in a consistent manner, typically through URIs (Uniform Resource Identifiers).
- **Resource-Based Architecture:** Everything is considered a resource (e.g., users, posts, comments), and these resources are manipulated using standard HTTP methods.

Examples of RESTful APIs

- **GET /users:** Retrieve a list of users.
- **POST /users:** Create a new user.

- **GET** /users/{id}: Retrieve a user by ID.
- **PUT** /users/{id}: Update a user by ID.
- **DELETE** /users/{id}: Delete a user by ID.

Best Practices and the Wrong vs. Right Way of Writing REST APIs

Designing RESTful APIs is mostly straightforward, but there are some common mistakes that you should avoid.

1. Using Verbs in URLs

Wrong Way:

```
GET /getAllBooks  
POST /createNewBook
```

Right Way:

```
GET /books  
POST /books
```

In RESTful design, the endpoint URL should represent a resource (a **noun**), not an action (a verb). The **HTTP method** already indicates the action to be performed.

2. Not Using Appropriate HTTP Methods

Wrong Way:

```
POST /books/123/update to update a book with ID 123.
```

Right Way:

```
PUT /books/123 to update the book directly.
```

Each HTTP method has a specific purpose. GET retrieves data, POST creates new resources, PUT updates existing resources, and DELETE removes resources. Using them correctly makes your API intuitive.

3. Not Using Proper Status Codes

Wrong Way:

```
200 OK for all responses.
```

Right Way:

```
200 OK for successful GET, POST, PUT, and DELETE.  
201 Created for successful POST.  
204 No Content for successful DELETE.  
404 Not Found for resource not found.
```

HTTP status codes are standardized codes that indicate the result of an HTTP request. Using them properly helps clients understand the outcome without parsing the response body.

4. Inconsistent Naming Conventions

Wrong Way:

Mixing singular and plural nouns in endpoints (`/book/123`, `/authors`).

Right Way:

Consistently using plural nouns (`/books/123`, `/authors`).

Consistency in your API design helps developers predict endpoint structures, reducing confusion and errors.

5. Not Implementing Pagination

Wrong Way:

Returning all records in a single response, which can be slow and resource-intensive.

Right Way:

Implementing pagination using query parameters like `?page=2&limit=50`.

Pagination divides responses into manageable chunks, improving performance and user experience.

Let's take look at a real example of a REST API design in the next section.